Lab 3

Execution Time, Memory Usage and Complexity Comparison

Field	Details
Name	Tamanna
Roll No.	2401201138
Course	BCA(AI & DS)
Section	A
Lab Name	Lab 3 - Smart Resource Allocation and Scheduling System using Greedy Algorithms

1. Objective

Measure the execution time and peak memory usage of the three greedy algorithms implemented in Lab 3 (Fractional Knapsack, Activity Selection and Huffman Coding) for growing input sizes, and prepare a complexity comparison table.

2. Measurement Method

- Execution time: time.perf_counter(); best (minimum) of 5 runs for each input size.
- Memory usage: tracemalloc peak allocation (in KB) during one run of the algorithm.
- Inputs: random data with a fixed seed (7). Knapsack capacity = half of the total weight; Huffman input = random lowercase text of length n.
- Input sizes n: 100, 500, 1000, 5000, 10000, 20000, 50000.

3. Measured Results

n	Knapsack Time (ms)	Knapsack Mem (KB)	Activity Time (ms)	Activity Mem (KB)	Huffman Time (ms)	Huffman Mem (KB)
100	0.017	1.4	0.009	1.9	0.024	8.4
500	0.094	21.3	0.056	11.9	0.041	13.8
1000	0.215	44.7	0.169	23.6	0.076	20.7
5000	1.953	231.9	1.387	117.1	0.288	72.2
10000	4.165	466.6	4.036	234.6	0.845	138.5
20000	9.062	935.4	9.349	468.9	1.368	270.8
50000	22.404	2341.5	19.459	1171.9	3.185	675.4

Note: absolute timings depend on the machine; the growth trend is what matters.

4. Complexity Comparison Table

Algorithm	Time Complexity	Space Complexity	Dominant Step
Fractional Knapsack	O(n log n)	O(n)	Sorting items by value/weight ratio
Activity Selection	O(n log n)	O(n)	Sorting activities by finish time
Huffman Coding	O(n log n)	O(n)	n-1 heap operations, each O(log n) (n = distinct characters)

5. Observations

- Knapsack and Activity Selection both sort their input, so their time grows slightly faster than linearly (n log n). From n = 10000 to 50000 the knapsack time went from 4.165 ms to 22.404 ms.
- Huffman Coding was the fastest in these runs because the heap holds only the distinct characters (at most 26 here), not all n symbols. Only counting and encoding scale with n.
- Peak memory of Knapsack and Activity Selection grows linearly with n, matching the O(n) space bound (the sorted copy of the input). Huffman memory grows linearly as well, mainly from the encoded bit string.
- All three algorithms are efficient and practical even for 50,000 inputs, taking only a few milliseconds.